Object Oriented Programming (CSEG1044)
Unit 02 – Lecture 01 Notes
Classes, Objects, and Encapsulation (Basics)

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**
- 7.1 Class fundamentals: fields, methods, state vs behavior . . . 3
- 7.2 Objects and reference semantics (mental model) . . . 4
- 7.3 Encapsulation: protect invariants . . . 4

8 Running Project Connection **4**

9 Common Mistakes and Confusions **5**

10 Worked Example: `Student` (encapsulation + references) **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **6**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Classes, Inheritance, Packages, and Interfaces
- **Main new ideas:** Class fundamentals: fields, methods, state vs behavior, Objects and reference semantics (mental model), Encapsulation: protect invariants
- **Course thread:** Use Classes, Objects, and Encapsulation (Basics) to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

2 Lecture Overview

In Unit 01 we wrote programs using variables, control flow, arrays, and strings. Now we start writing **real OOP code**: we will create classes and objects and learn how encapsulation makes programs safer.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Encapsulation principle**: Unit 01 introduced encapsulation conceptually; this lecture turns it into actual Java class design. Main reference: `Unit 01 / Lecture 01 slides/notes`.
- **Java class syntax**: You already saw the minimum class structure and now reuse it for reusable domain models. Main reference: `Unit 01 / Lecture 02 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Identify fields vs methods and explain state vs behavior.
- Explain references and aliasing using a small example.

- Apply encapsulation using `private` fields and validation methods.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Class fundamentals: fields, methods, state vs behavior* → *Objects and reference semantics (mental model)* → *Encapsulation: protect invariants.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Field**: data stored inside an object (state).
- **Method**: behavior (operations) of an object.
- **Reference**: a variable that points to an object (not the object itself).
- **Aliasing**: two references pointing to the same object.
- **Encapsulation**: keeping fields private and changing state through methods.
- **Access modifier**: `private`, default, `protected`, `public` (controls visibility).

7 Detailed Notes

7.1 Class fundamentals: fields, methods, state vs behavior

A **class** is a blueprint. It defines:

- **Fields** (what data is stored),
- **Methods** (what operations are allowed),
- and usually a **constructor** (how objects are created).

State vs behavior:

- State = current values in fields (e.g., `cgpa=8.2`).
- Behavior = operations that read/change state (e.g., `updateCgpa(...)`).

Rule of thumb. A class should have a clear responsibility. If it tries to do everything, it becomes hard to maintain.

7.2 Objects and reference semantics (mental model)

In Java, objects are created on the heap (conceptually), and variables hold **references** to objects. So assignment copies the reference (not the whole object).

Aliasing example:

- `Student a = new Student(...);`
- `Student b = a;` → now both `a` and `b` refer to the same object.
- Changes through `b` are visible through `a`.

Why it matters. Many bugs come from unexpected aliasing. Later, you will use immutability and defensive copying in some designs.

7.3 Encapsulation: protect invariants

Encapsulation is not only "hiding data"; it is mainly about **protecting invariants**. If fields are public, any code can set invalid values.

Example invariant: CGPA must be in range `0.0` to `10.0`.

Approach:

- Make fields `private`.
- Provide methods that validate inputs before updating the state.
- Expose only what users need (avoid unnecessary setters).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

8 Running Project Connection

Use Classes, Objects, and Encapsulation (Basics) to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Making every field public for convenience.
- Confusing an object reference with a copy of the object state.
- Thinking classes matter only inside main.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

10 Worked Example: `Student` (encapsulation + references)

Code

```java
public class Student {
    private final String rollNo;
    private final String name;
    private double cgpa; // invariant: 0.0 <= cgpa <= 10.0

    public Student(String rollNo, String name, double cgpa) {
        if (rollNo == null || rollNo.isBlank()) throw new IllegalArgumentException("rollNo
            ");
        if (name == null || name.isBlank()) throw new IllegalArgumentException("name");
        if (cgpa < 0.0 || cgpa > 10.0) throw new IllegalArgumentException("cgpa");
        this.rollNo = rollNo;
        this.name = name;
        this.cgpa = cgpa;
    }

    public double getCgpa() { return cgpa; }

    public void updateCgpa(double newCgpa) {
        if (newCgpa < 0.0 || newCgpa > 10.0) throw new IllegalArgumentException("cgpa");
        this.cgpa = newCgpa;
    }

    public String summary() {
        return rollNo + \" - \" + name + \" (CGPA=\" + cgpa + \")\";
    }
}
```

Aliasing demo (main)

```java
public class Demo {
```

```java
    public static void main(String[] args) {
        Student a = new Student("21CS1001", "Asha", 8.2);
        Student b = a; // aliasing
        b.updateCgpa(9.0); // changes same object
        System.out.println(a.getCgpa()); // prints 9.0
    }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Class fundamentals: fields, methods, state vs behavior* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. Two references `a` and `b` point to the same object. If `b` modifies it, does `a` see the change?

   Answer: yes (aliasing).

2. Why is it risky to keep fields `public`?

   Answer: any code can put the object into an invalid state, breaking invariants.

3. Write one invariant for a `BankAccount` class.

   Sample: `balance >= 0`.

13 Self-Study Practice (no solutions)

1. Design a `LibraryBook` class with an invariant (e.g., cannot issue if already issued).
2. Write a small program that shows aliasing using a class of your choice.
3. Refactor a "public fields" class into an encapsulated design with validation methods.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.

- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. What is encapsulation and why do we use it?

Sample answer: Encapsulation means keeping internal data private and changing it only through methods. It helps protect invariants and prevents accidental misuse, making code easier to maintain.

Q2. What is aliasing?

Sample answer: Aliasing happens when two references point to the same object. Updates through one reference are visible through the other.

Q3. What problem does encapsulation solve beyond hiding fields?

Sample answer: It protects invariants and controls how state changes through a safer public interface.